

Interactive SPH Fluid Simulation

with Two-Way Rigid Body Coupling

Anawat Bumroongsri (6632245521)

Sarit Pasiphol (6632234621)

1 Introduction

Real-time fluid simulation is computationally demanding, requiring a balance between realism and performance. This project implements a particle-based approach using **Smoothed Particle Hydrodynamics (SPH)**, where fluids are modeled as discrete particles with physical properties such as density, pressure, and velocity. Unlike grid-based methods, SPH naturally supports free surfaces and large deformations.

The system also features **two-way coupling** between fluids and rigid bodies, enabling realistic interactions such as floating, sinking, and fluid displacement. Implemented from scratch in **C++17 with OpenGL**, the project includes a custom SPH solver and spatial hashing, without relying on external simulation frameworks.

2 Project Overview

2.1 Goals and Scope

The primary goals of this project are:

- Implement a stable, interactive SPH fluid solver following Müller et al. [1].
- Achieve fast neighborhood search via a uniform spatial hash grid.
- Implement two-way fluid-rigid body coupling following Akinci et al. [2].
- Provide a real-time parameter interface using Dear ImGui.

2.2 System Architecture

The project is organized into three modules that map onto the 4-week development timeline:

Module	Files	Responsibility
Core	Window.h/cpp, Renderer.h/cpp	OpenGL context, window management, draw calls
Simulation	SPHSolver.h/cpp, SpatialHash.h/cpp, SmoothKernel.h, Particle.h, ParticleUtils.h, SimParams.h	SPH physics update, kernel functions, spatial neighborhood search, simulation parameters
Simulation	RigidBody.h, RigidBodyUtils.h	Boundary particle sampling, two-way force exchange, rigid body dynamics
UI	DebugUI.h/cpp	Dear ImGui panels, real-time parameter tuning
Entry Point	main.cpp	Application entry, main loop orchestration

Each simulation step follows the pipeline:

Neighbor Search $\rightarrow$ Density / Pressure $\rightarrow$ Force Accumulation $\rightarrow$ Integration $\rightarrow$ Render

3 Implementation

3.1 SPH Fluid Solver

3.1.1 Particle Representation

Each fluid particle i stores position $\mathbf{x}_i$, velocity $\mathbf{v}_i$, density ρ_i , and pressure p_i . The core of SPH is the interpolation of a scalar or vector quantity A at any point $\mathbf{x}$:

$$A(\mathbf{x}) = \sum_j m_j \frac{A_j}{\rho_j} W(\mathbf{x} - \mathbf{x}_j, h) \quad (1)$$

where m_j is the particle mass, h is the smoothing radius (kernel support), and W is a smoothing kernel function. By choosing W carefully, spatial derivatives of A can be computed analytically from derivatives of W , avoiding any mesh.

3.1.2 Smoothing Kernels

Following Müller et al. [1], three purpose-built kernels are used, each optimized for a specific force term:

Poly6 — Density:

$$W_{\text{poly6}}(\mathbf{r}, h) = \frac{315}{64\pi h^9} \begin{cases} (h^2 - |\mathbf{r}|^2)^3 & 0 \leq |\mathbf{r}| \leq h \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

Spiky — Pressure:

$$\nabla W_{\text{spiky}}(\mathbf{r}, h) = -\frac{45}{\pi h^6} \cdot \frac{\mathbf{r}}{|\mathbf{r}|} \begin{cases} (h - |\mathbf{r}|)^2 & 0 \leq |\mathbf{r}| \leq h \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

Viscosity — Friction:

$$\nabla^2 W_{\text{visc}}(\mathbf{r}, h) = \frac{45}{\pi h^6} \begin{cases} (h - |\mathbf{r}|) & 0 \leq |\mathbf{r}| \leq h \\ 0 & \text{otherwise} \end{cases} \quad (4)$$

3.1.3 Density and Pressure

Density at particle i is accumulated from neighbors:

$$\rho_i = \sum_j m_j W_{\text{poly6}}(\mathbf{x}_i - \mathbf{x}_j, h) \quad (5)$$

Pressure is recovered via the equation of state:

$$p_i = k (\rho_i - \rho_0) \quad (6)$$

where ρ_0 is the rest density and k is a stiffness constant. Larger k yields stiffer fluid at the cost of requiring smaller timesteps for stability.

3.1.4 Force Computation

Three force contributions are summed per particle:

$$\mathbf{f}_i^{\text{pressure}} = - \sum_j m_j \frac{p_i + p_j}{2\rho_j} \nabla W_{\text{spiky}}(\mathbf{x}_i - \mathbf{x}_j, h) \quad (7)$$

$$\mathbf{f}_i^{\text{viscosity}} = \mu \sum_j m_j \frac{\mathbf{v}_j - \mathbf{v}_i}{\rho_j} \nabla^2 W_{\text{visc}}(\mathbf{x}_i - \mathbf{x}_j, h) \quad (8)$$

$$\mathbf{f}_i^{\text{gravity}} = \rho_i \mathbf{g} \quad (9)$$

The symmetric pressure formulation (averaging $p_i + p_j$) conserves linear momentum across particle pairs.

3.1.5 Neighborhood Search

A brute-force $O(n^2)$ search is infeasible at interactive rates. We implement a **uniform spatial hash grid** where each cell has side length h . Particles are bucketed by their integer cell coordinate, hashed with large primes:

$$\text{key} = \lfloor x/h \rfloor \cdot p_1 \oplus \lfloor y/h \rfloor \cdot p_2 \oplus \lfloor z/h \rfloor \cdot p_3$$

Only the 27 adjacent cells need to be queried per particle, reducing average complexity to $O(1)$ per particle per timestep.

3.2 Two-Way Rigid Body Coupling

We implement a sample-based penalty coupling: rigid bodies store surface samples $\mathbf{s}_k$ (transformed to world space each frame) and each sample uses a uniform weight $w_k = 1/N_{\text{samples}}$. For each sample the solver queries fluid neighbors within the smoothing radius h . For a particle at $\mathbf{x}$ let $\mathbf{d} = \mathbf{x} - \mathbf{s}$, $r = \|\mathbf{d}\|$, and unit normal $\mathbf{n} = \mathbf{d}/r$; define penetration $\delta = h - r$ when $0 < r < h$.

Per-contact acceleration combines a penetration-based stiffness and normal damping. Compactly:

$$a = \text{clamp} \left((k_s \frac{\delta}{h} + k_d \max(0, -v_n)) S(r) w_k D(\rho), a_{\max} \right), \quad (10)$$

where k_s and k_d are ‘Sim::params.rigidBoundaryStiffness’ and ‘Sim::params.rigidBoundaryDamping’, $v_n = \mathbf{n} \cdot (\mathbf{v} - \mathbf{v}_s)$, $S(r) = (h - r)/h$, and $D(\rho)$ scales by local density (clamped). The particle acceleration is incremented by $a\mathbf{n}$ and the opposite force ($-m_p a\mathbf{n}$) is applied to the rigid via ‘RigidBody::applyForceAtPoint’.

Time integration follows a short two-phase pattern (as in ‘SPHSolver::step’): accumulate sample forces onto rigid bodies, integrate rigids (gravity + Newton–Euler), then recompute and apply updated sample responses to fluid particles. The method keeps momentum exchange explicit and uses per-step clamping, density scaling and spatial falloff to maintain stability and interactive performance.

3.2.1 Comparison with Akinici et al.

In contrast to Akinici et al. (2012), which explicitly computes per-sample volumes Ψ_k and includes boundary particles in the density summation, our implementation uses a penalty-style response computed from sample penetration and velocity damping (see ‘SPHSolver::applyRigidBoundaryCoupling’). The practical differences are: we do not modify fluid density with boundary contributions, we use a simple per-sample weight w_k instead of a precomputed Ψ_k , and the contact response uses a linear spatial falloff and clamped acceleration instead of direct kernel-weighted pressure/viscous forces.

This design is intentional: the penalty+damping approach is simpler to implement, cheaper to evaluate per timestep, and—critically—offers predictable stability controls (per-step clamping, density-based scaling). For an interactive demo and parameter exploration this tradeoff favours robustness and low latency; it still conserves momentum via equal-and-opposite force accumulation on the rigid bodies. If higher-fidelity boundary density enforcement is required later, computing Ψ_k and adding boundary contributions to ‘computeDensityPressure()’ is a straightforward extension.

4 Member Contributions

Member	Contributions
Anawat Bumroongsri (6632245521)	• Set up the CMake build system and OpenGL/GLFW/GLAD project skeleton. • Implemented the uniform spatial hash grid (<code>SpatialHash.h/cpp</code>). • Implemented the SPH solver: all three kernels, density, pressure, and viscosity forces (<code>SPHSolver.h/cpp</code>). • Implemented particle rendering with OpenGL VBOs and GLSL shaders.
Sarit Pasiphol (6632234621)	• Designed and implemented boundary particle sampling for static walls and dynamic rigid bodies (<code>RigidBody.h/cpp</code>). • Implemented the two-way coupling force exchange following Akinci et al. • Implemented rigid body dynamics integration (translation and rotation). • Built the Dear ImGui debug panel (<code>DebugUI.h/cpp</code>) for live parameter control. • Tuned simulation parameters for stability and wrote the project report.

References

- [1] M. Müller, D. Charypar, and M. Gross, “Particle-based fluid simulation for interactive applications,” in *Proc. ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, 2003.
- [2] G. Akinci, M. Ihmsen, N. Akinci, B. Solenthaler, and M. Teschner, “Versatile rigid-fluid coupling for incompressible SPH,” *ACM Transactions on Graphics (TOG)*, vol. 31, no. 4, 2012.
- [3] M. Becker, M. Ihmsen, and M. Teschner, “Direct forcing for Lagrangian fluids,” *IEEE Trans. Visualization and Computer Graphics*, 2009.
- [4] M. Ihmsen, J. Orthmann, B. Solenthaler, A. Kolb, and M. Teschner, “SPH fluids in computer graphics,” *Eurographics State of the Art Report (STAR)*, 2014.